

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Página Web Productora 2 (SCRUM-20)

1. CONTEXTO

Se requiere una página web de presentación corporativa para una productora, compuesta por 4 secciones (Home, Servicios, Quiénes Somos, Contacto). El stack está explícitamente restringido a HTML5 + CSS + JS puro. No existe backend propio, no hay base de datos, y el formulario de contacto debe operar sin almacenamiento de datos. El equipo de diseño aún no ha entregado assets finales, por lo que se usarán placeholders de URLs externas obligatoriamente.

2. DECISIÓN ARQUITECTÓNICA

Arquitectura elegida: Sitio Estático Multi-Página (MPA Clásico) — HTML/CSS/JS Vanilla

Se implementará como un conjunto de archivos HTML estáticos interconectados mediante navegación tradicional (`<a href>`), con un único archivo CSS global y un único archivo JS modular. El formulario de contacto utilizará un servicio de terceros (Formspree o EmailJS) mediante fetch/XHR desde el frontend, eliminando la necesidad de cualquier backend.

Estructura de archivos: 4 páginas HTML independientes (index.html, servicios.html, quienes-somos.html, contacto.html) que comparten un único `styles.css` y un único `main.js`. Los componentes repetidos (navbar, footer) se inyectarán dinámicamente via JS para evitar duplicación de código HTML.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS

- **¿Por qué NO SPA (React/Vue/Angular)?** El requerimiento explícita HTML5+CSS+JS solamente. Una SPA añadiría un bundler (Webpack/Vite), un framework JS, y complejidad de routing innecesaria para 4 páginas estáticas. YAGNI: no se necesita estado reactivo ni componentes dinámicos complejos.
- **¿Por qué NO un generador estático (Jekyll/Hugo/Eleventy)?** Añadiría una dependencia de build pipeline (Node.js, Ruby) que el requerimiento no contempla y que el cliente no podría mantener sin asistencia técnica. El stack puro HTML/CSS/JS es directamente deployable en cualquier hosting sin proceso de compilación.
- **¿Por qué NO backend propio para el formulario?** El requerimiento explícitamente excluye integración con sistemas internos y prohíbe almacenar datos del formulario. Un servicio de terceros (Formspree free tier) resuelve el envío de email sin infraestructura adicional, cumpliendo la restricción de máximo 2 componentes externos (CDN para assets + Formspree para formulario).
- **¿Por qué NO localStorage/sessionStorage para el formulario?** El requerimiento de seguridad prohíbe almacenar datos sensibles. El formulario envía directamente via fetch a Formspree y descarta los datos del DOM tras confirmación.
- **¿Por qué NO CSS Framework (Bootstrap/Tailwind)?** Se puede lograr diseño responsivo moderno con CSS Grid + Flexbox + Custom Properties nativas. Añadir Bootstrap vía CDN introduce ~150KB adicionales y clases que el desarrollador debe conocer. CSS vanilla es más mantenible para este scope.
- **Internacionalización (ES/EN):** Se implementará con un switcher JS que intercambia atributos `data-i18n` en el DOM, cargando un objeto de traducciones desde `i18n.js`. No se requiere librería externa.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (Bajo):** La inyección de navbar/footer via JS puede causar un flash de contenido sin estilos (FOUC) en conexiones lentas. Mitigación: el JS de componentes se carga de forma síncrona en el `<head>` con `defer`.
- **Riesgo 2 (Medio):** Formspree free tier tiene límite de 50 envíos/mes. Si el volumen de contacto supera esto,

se requiere plan de pago (~\$10/mes). Mitigación: documentar en README y alertar al cliente.

- **Riesgo 3 (Bajo):** Sin CMS integrado, las actualizaciones de contenido requieren edición directa de HTML. El requerimiento menciona integración CMS pero el stack HTML puro lo hace incompatible sin un paso adicional. Decisión: se estructura el HTML con comentarios claros `<!-- EDITABLE: Actualizar contenido aquí -->` para facilitar edición manual hasta que se defina el CMS.

- **Riesgo 4 (Bajo):** La protección XSS se implementa sanitizando inputs via JS antes del envío (encode de caracteres especiales). Sin backend propio, la superficie de ataque SQL Injection es inexistente (no hay DB). La protección OWASP se aplica en el lado cliente únicamente.

5. CONTRATO DE INTEGRACIÓN

- Servicio de formulario: Formspree endpoint `https://formspree.io/f/{FORM_ID}` — el FORM_ID debe configurarse en `js/config.js`

- Imágenes: Todas las `<img>` apuntan a `https://placeholder.co/{W}x{H}` o `https://picsum.photos/{W}/{H}` hasta integración final

- Fuentes: Google Fonts via CDN (Inter + Playfair Display)

- Sin archivos binarios locales (.png, .jpg, .webp) en el repositorio

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- servicios.html | Estado: APROBADO | Iteraciones: 3
- quienes-somos.html | Estado: APROBADO | Iteraciones: 2
- contacto.html | Estado: APROBADO | Iteraciones: 2
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- js/config.js | Estado: APROBADO | Iteraciones: 1
- js/components.js | Estado: APROBADO | Iteraciones: 2
- js/i18n.js | Estado: APROBADO | Iteraciones: 1
- js/contact-form.js | Estado: APROBADO | Iteraciones: 1
- js/main.js | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 147,538

Tokens de salida (output): 56,669

Total tokens: 204,207

Horas-hombre estimadas: ~19.6 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"4,166 input | !1,876 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: !"29,085 input | !16,499 output | 11 llamadas

FRONTEND-DESARROLLO: !"38,506 input | !35,562 output | 16 llamadas

QA-VALIDACION: !"75,781 input | !2,732 output | 16 llamadas